
CTBench: Evaluating Troubleshooting Capabilities of AI Agents in Realistic Telecom Network Operations

Xingyu Yan*, Tingting Dai*, Antonio De Domenico[†], Mohamed Sana[†], Nicola Piovesan[†],
 Changchang Li*, Bowen Liu*, Kun Jiang*, Mengjie Zhang*, Dingcheng Shan*,
 Jing-Cheng Pang*, Chenwei Wu*, Sijie Wu*, Lianying Chao*, Haoran Cai*,
 Jiantao Ye*, Xubin Li*, Simon Mark Lucas⁺, Xin Chen*

*Huawei Technologies, China

[†]Paris Research Center, Huawei Technologies, Boulogne-Billancourt, France

⁺Queen Mary University of London

Abstract

Agents are increasingly considered for automating network operations and maintenance, where engineers must diagnose network faults, optimize configurations to enhance services, and reduce operational costs while acting under strict constraints. However, existing evaluations fail to accurately model real network characteristics or assess agents under partially observable telecom environments with diverse vendors, devices, protocols, and interfaces. In this paper, we introduce CTBench, a public benchmark for assessing whether an agent behaves like a competent telecom troubleshooting engineer. CTBench focuses on root cause analysis and path restoration. Each task is constructed by experts and annotated with rich task metadata, including golden evidence steps. CTBench uses expert-grounded metrics that evaluate both final answers and the diagnostic evidence. Experiments with representative harness-model combinations show that state-of-the-art agents perform very well at identifying endpoints in path-restoration tasks but, more generally, underperform in root cause analysis. In particular, agents struggle with interface state, link-layer, service-management, and other operational faults. Most importantly, even when agents produce plausible or correct final answers, they often fail to provide the evidence-grounded diagnoses required in operational practice. Our results further show that path restoration is generally more resource expensive, yet larger resource usage does not necessarily translate into better diagnosis.

1 Introduction

Network operations and maintenance (NetO&M) is a demanding operational domain where engineers must interpret heterogeneous telemetry, issue device-specific commands, reconstruct service paths, and diagnose faults across multiple network layers. Unlike many static question-answering settings, NetO&M requires sequential decision making: an operator must form hypotheses, select diagnostic actions, inspect command outputs, rule out alternative explanations, and produce an evidence-supported conclusion. These requirements make telecom troubleshooting a natural and challenging testbed for evaluating agentic AI systems.

Recent advances in large language model (LLM) agents have demonstrated strong capabilities in multi-step reasoning, tool use, and interactive problem solving [18, 17, 19]. In communication technology, such agents could assist with root cause analysis, path reconstruction, configuration checking, and closed-loop remediation.

Deployment remains difficult because real networks are partially observable, carry cascading cross-layer faults, and expose multi-vendor command interfaces under strict operational safety requirements.

Benchmark	RCA	Path Restoration	Partial Observability	Network Heterogeneity	Expert-Annotated Evidence	Multi-Dimensional Metrics	Real Equipment
NIKA	✓	×	×	×	×	✓	×
NetArena	✓	×	×	×	×	✓	×
NetAgentBench	✓	×	×	×	×	✓	×
CTBench (ours)	✓	✓	✓	✓	✓	✓	✓

Table 1: Conceptual comparison between CTBench and existing telecom benchmarks.

Existing telecom benchmarks do not reproduce these conditions. They assume full observability, use single-domain topologies, and do not focus on real network equipment. In addition, these benchmarks cannot distinguish an agent that is unable to localize a fault in a malfunctioning equipment from one that returns a correct label without supporting evidence. For operational NetO&M, this process-level transparency is a prerequisite for trust, since a diagnosis must be justified before it can be acted on. Furthermore, a realistic benchmark should assess the capabilities of any tool-using agent in a professional setting: acting under partial observability, adapting across heterogeneous interfaces, and attributing network issues across multi-hop propagation rather than stopping at the first visible symptom.

We introduce CTBench, an agentic benchmark for realistic telecom troubleshooting, covering two operational task families. In RCA tasks, the agent must identify the affected node and object and assign a normalized root-cause label. In path-restoration tasks, the agent must identify source and destination endpoints and reconstruct the forwarding path. The benchmark contains 234 expert-curated tasks, 126 for RCA and 108 for path restoration.

Evaluation in CTBench is grounded in the agent reasoning traces and expert practice rather than final answers alone. Each task carries an expert-normalized answer together with expert-validated golden evidence steps that capture the observations and diagnostic actions required to solve it. Tasks are built by 15 senior telecom experts and validated by an independent expert who solves the tasks without access to ground truths. Each task is further annotated with metadata describing its structural difficulty, including evidence observability, vendor and device heterogeneity and protocol complexity.

Our contributions are as follows:

- We introduce CTBench, a public benchmark for evaluating agentic troubleshooting in telecom NetO&M, covering 234 expert-curated RCA and path-restoration tasks.
- We define telecom-expert-grounded capability metrics that separately evaluate localization, root-cause identification, path restoration, evidence acquisition, observability robustness, heterogeneous device/vendor handling, and resource usage.
- We provide a task metadata schema covering observability, device/vendor heterogeneity, protocol complexity, root-cause count, restored path count, fault-propagation chains, golden solution length, and root-cause categories, enabling fine-grained diagnostic analysis beyond aggregate accuracy.
- We evaluate representative agent-model combinations and show that current agents exhibit substantial gaps in evidence-grounded diagnosis, partial-observability reasoning, and robust handling of heterogeneous telecom environments.

2 Related Work

2.1 Benchmarks for LLM Agent

General-purpose LLM agent benchmarks [9, 5, 6, 10, 2] focus on evaluating multi-step reasoning, tool use, web navigation, software engineering, and interactive planning ability. These benchmarks have been useful for measuring whether agents can decompose tasks, call tools, and synthesize intermediate observations. However, most general agent benchmarks do not capture the specific operational constraints of telecom NetO&M: vendor-specific command syntax, network topology reasoning, control-plane and data-plane interaction, partial device access, and evidence-grounded fault isolation. As a result, high performance on general agent benchmarks does not necessarily imply operational competence in telecom troubleshooting.

2.2 Benchmarks for Network and Telecom

Several benchmarks target networking and telecom domains. Knowledge-oriented benchmarks such as TeleQnA [8] and ORAN-Bench [4] evaluate whether models understand telecom specifications or standards. Troubleshooting-oriented benchmarks such as NIKA [16], TeleLogs [12], WirelessAgent++ [13], TelcoAgent-Bench [1], NetAgentBench [14], and NetArena [20] move closer to operational network reasoning by introducing diagnostic cases, configuration tasks, and network automation scenarios [7]. These efforts provide important foundations for evaluating LLMs and agents in communication technology.

Nevertheless, existing benchmarks often leave at least one major operational gap: they may assume full observability, focus on single-domain topologies, use standardized interfaces, or evaluate only final answers. CTBench complements prior work by emphasizing expert-grounded trajectory evaluation, partial observability, heterogeneous network environments, and evidence acquisition. Table 1 summarizes the conceptual comparison between CTBench and existing telecom benchmarks.

3 CTBench Benchmark

This section formalizes the tasks in CTBench, and presents the associated datasets.

3.1 Task Formulation

CTBench evaluates telecom troubleshooting as an interactive decision-making problem. Each benchmark instance is a task instruction q_τ , where $\tau \in \{\text{RCA}, \text{Path}\}$ denotes the task type. Let π_θ denote an evaluated agent, including both the agent harness and the underlying model with parameters θ .

At each interaction turn t , the agent selects a diagnostic action conditioned on the task instruction and the interaction history:

$$a_t \sim \pi_\theta(\cdot \mid q_\tau, h_{t-1}), \quad (1)$$

where a_t is an action such as executing a device command or querying an allowed telemetry source, and $h_{t-1} = (a_1, o_1, \dots, a_{t-1}, o_{t-1})$ is the history of previous actions and observations. The task environment $\mathcal{E}_\tau$ executes the action and returns an observation $o_t = \mathcal{E}_\tau(q_\tau, h_{t-1}, a_t)$, where o_t may contain command outputs, configuration parameters, routing information, interface states, policy rules, or an error message if the action is invalid or unavailable. The full interaction trajectory is $\mathcal{T}(q_\tau) = (q_\tau, a_1, o_1, a_2, o_2, \dots, a_T, o_T)$, from which, the agent predicted answer $\hat{y}_\tau$ is decoded.

Root Cause Analysis. An RCA task provides a problem description and an interactive diagnostic interface. The agent must issue permitted queries or commands, inspect the returned observations, and output a normalized diagnosis. For RCA tasks, the target answer is a set of normalized root-cause triples:

$$y_{\text{RCA}}^* = \{(n_i, o_i, c_i)\}_{i=1}^m, \quad (2)$$

where n_i is the affected node, o_i is the affected object, and c_i is the normalized root-cause label. The affected object may be an interface, route, tunnel, access control list rule, network address translation policy, virtual private network instance, service object, or another telecom component.

Path Restoration. A path-restoration task asks the agent to reconstruct a service forwarding path. For path-restoration tasks, the target answer is a reconstructed forwarding path comprising an ordered sequence of path elements p_j :

$$y_{\text{Path}}^* = \{p_1, p_2, \dots, p_k\}, \quad (3)$$

Each path element p_j may include a node, interface, next hop, policy, or branch decision.

3.2 Dataset Construction

CTBench is constructed through an expert-in-the-loop process. Candidate tasks are abstracted from realistic telecom maintenance cases and sanitized to remove sensitive production information. 15 senior telecom domain experts with an average of 20 years of professional experience define the

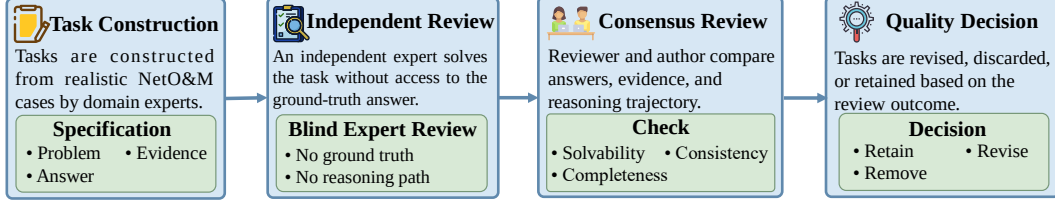


Figure 1: Process of expert-involved data quality review for CTBench. Candidate tasks are sanitized, independently reviewed by telecom experts, checked for answer and evidence consistency, revised when necessary, and retained only after consensus on solvability and operational validity.

problem statement, available device outputs, permitted command interface, standard answer, and task-side metadata. The current benchmark contains 126 RCA tasks and 108 path-restoration tasks.

To ensure task validity, each candidate task is reviewed by an independent telecom expert who is not involved in the original construction. The reviewer receives the problem description and permitted interface but not the ground-truth answer or intended trajectory. The reviewer independently solves the task, after which the task author and reviewer compare final answer, supporting evidence, and reasoning trajectory. Tasks are retained only when experts reach consensus on solvability, answer completeness, evidence consistency, and alignment with the intended network behavior. Ambiguous, underspecified, or unverifiable cases are revised or discarded.

Figure 1 summarizes this expert-involved data quality workflow, from task construction and sanitization to independent expert review, consensus checking, revision, and final release.

3.3 Golden Evidence Annotation

A key feature of CTBench is that it evaluates not only the final answer $\hat{y}_\tau$, but also the diagnostic process used to obtain it. Each task is associated with an expert-validated golden solution represented as a set of golden actions:

$$\mathcal{A}^*(q_\tau) = \{a_1^*, a_2^*, \dots, a_{T^*}^*\}, \quad (4)$$

where each a_i^* denotes a key diagnostic action that a telecom expert considers necessary for solving the task and T^* is the total number of steps involved.

In CTBench, golden actions are defined from expert trajectories and therefore provide a deterministic reference for evaluating whether an agent can follow the key operational steps used by telecom experts.

For RCA, golden actions capture the diagnostic checks needed to localize and justify the root cause. For path restoration, they capture the checks needed to reconstruct the forwarding path, including endpoint reachability, hop adjacency, routing decisions, interface transitions, policy constraints, tunnel state, or branch behavior. Concrete annotation examples are provided in the supplementary material.

3.4 Task Metadata

CTBench annotates tasks with metadata that enables fine-grained analysis of agent capability.

Evidence Observability. For RCA tasks, CTBench defines two levels of evidence observability, summarized in Table 2. This label, denoted by $O(q_\tau) \in \{O_1, O_2\}$, indicates whether a fault evidence can be obtained directly through a permitted diagnostic command, or whether the agent must infer the fault from partial observations obtained through indirect commands.

Root-Cause Categories. Each RCA task is assigned to one or more root-cause categories. We define the categorization taxonomy at the level of individual root causes rather than at the task level: a single troubleshooting scenario may involve multiple underlying faults and can therefore be associated with multiple categories. Table 3 summarizes the resulting five-category taxonomy.

Label	Observability	Explanation
O1	Full	The agent can obtain decisive evidence of the fault by directly implementing a permitted diagnostic command.
O2	Partial	The direct diagnostic command is unavailable or fails. Therefore, the agent must rely on indirect commands that expose only partial observations, requiring additional reasoning and cross-checking to identify the correct root cause.

Table 2: Observability levels in CTBench.

Label	Category	Distribution
C1	Interface State and Link-Layer Faults	28.04%
C2	Security, Network Address Translation (NAT), and Edge Access Control	24.30%
C3	Routing Protocol and Policy Control	16.82%
C4	High Availability and Reliability Mechanisms	15.89%
C5	Service, Management, and Other Operational Faults	14.95%

Table 3: RCA category taxonomy and gold-entry distribution.

Root Cause count. For RCA tasks, CTBench records the number of independent gold root causes, denoted by $N_r(q_\tau)$. This metadata captures whether a troubleshooting scenario requires identifying a single failure or multiple simultaneous faults. Tasks with multiple root causes are structurally harder because the agent must distinguish independent causes from downstream symptoms and avoid returning an incomplete or overly broad diagnosis.

Fault-propagation chain. For RCA tasks, CTBench annotates the fault-propagation-chain length $N_c(q_\tau)$ when applicable. This metadata describes the causal path from the latent root cause to intermediate network states and finally to the observed symptom. Longer or more indirect propagation chains increase diagnostic difficulty because the agent must reason beyond the first visible symptom and identify the underlying cause.

Restored path count. For path-restoration tasks, CTBench records the number of restored forwarding paths in the gold answer, denoted by $N_{\text{path}}(q_\tau)$. A task may require reconstructing a single path or multiple paths caused by branching, redundancy, load balancing, or service-specific forwarding behavior. Multi-path cases are more difficult because the agent must restore path multiplicity rather than only one route.

Protocol complexity. For both RCA and path-restoration tasks, CTBench records protocol complexity, denoted by $N_p(q_\tau)$. This metadata captures the number and interaction depth of forwarding, control-plane, policy, redundancy, and service mechanisms involved in solving the task. Higher protocol complexity indicates that the correct answer depends on reasoning across multiple interacting mechanisms rather than inspecting a single local state.

Network Heterogeneity. Network heterogeneity captures the number of distinct vendor environments $N_v(q_\tau)$ and device types $N_d(q_\tau)$ the agent must handle. As shown in Table 4, we assign to each task a network heterogeneity level comparing the maximum between vendor heterogeneity and device-type heterogeneity $h_L(q_\tau) = \max(N_v(q_\tau), N_d(q_\tau))$ with task-calibrated thresholds for RCA and path-restoration tasks. Table 4 summarizes the reporting levels.

Golden Solution Length. For both task types, CTBench records the golden solution length $T^*(q_\tau) = |\mathcal{A}^*(q_\tau)|$, defined as the number of key evidence steps required to solve the task. This metadata captures the procedural burden of the task: a longer golden path indicates that the agent must collect, connect, and use more pieces of evidence before reaching a justified answer.

Using Metadata to Characterize Task Difficulty. These metadata fields allow CTBench to characterize task difficulty as a structural property of the task. Evidence observability captures information availability; root-cause count and restored path count capture answer multiplicity; fault-propagation chain length captures causal depth; protocol complexity captures cross-layer reasoning

Label	Name	RCA	Path Restoration
H-Low	Low heterogeneity	$h_L = 1$	$h_L \leq 6$
H-High	High heterogeneity	$h_L \geq 2$	$h_L > 6$

Table 4: Network heterogeneity levels. The levels are defined based on the number of vendors and devices type involved in the solution.

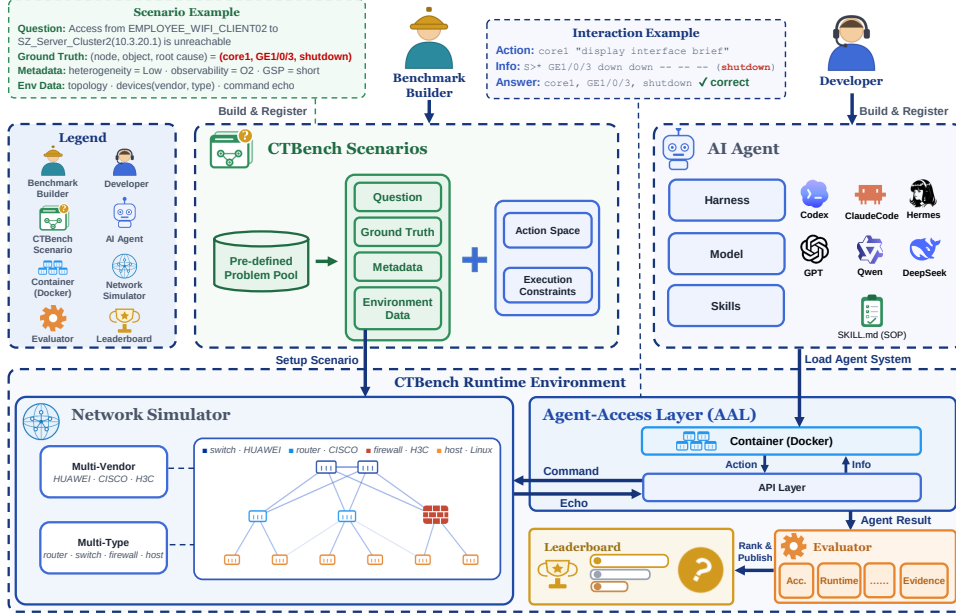


Figure 2: CTBench Automatic Evaluation framework.

burden; device/vendor heterogeneity captures semantic and operational diversity; and golden solution length captures the number of expert-required diagnostic steps. For RCA tasks, these dimensions describe the difficulty of localizing, identifying, and justifying root causes. For path-restoration tasks, difficulty is instead driven by endpoint identification, path multiplicity, forwarding structure, protocol interactions, heterogeneity, and the length of the evidence path. We use these annotations both for fine-grained reporting and for analyzing which operational conditions most challenge current agents.

4 Evaluation Protocol

Figure 2 illustrates the overall evaluation framework of CTBench. Given a task instance q_τ and an agent trajectory $\mathcal{T}(q_\tau)$, the evaluator compares the predicted answer $\hat{y}_\tau$ and collected evidence $\hat{\mathcal{A}}(q_\tau)$ against the ground truth y_τ^* and the related expert-annotated metadata $\mathcal{A}^*(q_\tau)$. In addition to the average task accuracy $\text{Acc} = \mathbb{E}[\mathbb{1}(y_\tau^* = \hat{y}_\tau)]$, CTBench reports task-specific capability metrics including cost and efficiency metrics.

4.1 RCA Capability Metrics

For RCA, CTBench defines three complementary capabilities.

RCA Localization. RCA Localization measures the agent capability to identify where the problem occurs, including the affected node, interface or service object. Let $L^* = \{(n_i, o_i)\}_{i=1}^m$ denote the gold set of affected node-object pairs extracted from y_{RCA}^* and $\hat{L}$ the predicted set extracted from $\hat{y}_{\text{RCA}}$. We compute the RCA localization score (RCA-Loc) as the intersection over union (IoU) between the predicted and gold localization sets:

$$\text{RCA-Loc} = \text{IoU}(\hat{L}, L^*), \quad (5)$$

where $\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$.

RCA Identification. RCA Identification measures the agent capability to identify the correct root-cause label, such as interface down, missing route or network misconfiguration. Let $C^* = \{c_i\}_{i=1}^m$ denote the gold set of normalized root-cause labels and $\hat{C}$ the predicted set. Similar to RCA-Loc, we compute the RCA Identification score (RCA-ID) as the IoU between the predicted and gold root-cause label sets:

$$\text{RCA-ID} = \text{IoU}(\hat{C}, C^*). \quad (6)$$

RCA Evidence. RCA Evidence quantitatively assesses whether the agent collects relevant key diagnostic evidence needed to identify the root cause. To do so, we compare the agent actions with the expert golden actions using the F1 score. Specifically, let $\mathcal{A}_{\text{RCA}}^*$ and $\hat{\mathcal{A}}_{\text{RCA}}$ denote the golden evidence and the agent steps, respectively. We compute:

$$\text{RCA-Evidence} = \text{F1}(\hat{\mathcal{A}}_{\text{RCA}}, \mathcal{A}_{\text{RCA}}^*). \quad (7)$$

A step is considered covered only if the agent queries the correct device with the correct command to retrieve the relevant observation required for diagnosing the problem.

Remark (Hierarchical Scoring). *The main RCA metrics above use strict set matching to accurately assess agent capabilities. To support near-miss analysis, we additionally define hierarchical RCA scoring and topology-aware node similarity as supplementary metrics. These metrics are not used as primary correctness measures; instead, they help distinguish close errors from completely unrelated predictions. The complete taxonomy, similarity rules, procedure, and associated results are provided in the supplementary material.*

4.2 Path Restoration Capability Metrics

Similar to RCA, CTBench defines three capability metrics.

Path Localization. Path localization measures whether the agent correctly identifies the relevant endpoints involved in the path-restoration task. Let P_{end}^* and $\hat{P}_{\text{end}}$ be the gold and predicted endpoint sets. We compute path localization score as the IoU between the gold and the predicted sets:

$$\text{Path-Loc} = \text{IoU}(\hat{P}_{\text{end}}, P_{\text{end}}^*). \quad (8)$$

Path Restoration. Path Restoration measures whether the agent reconstructs the correct forwarding path. Let P^* denote the reference path edge sets and $\hat{P}$ the predicted path edge sets. We compute the path restoration score as IoU between predicted and reference path edge sets:

$$\text{Path-Res} = \text{IoU}(\hat{P}, P^*). \quad (9)$$

Path Evidence. Path Evidence measures whether the agent collects and uses the key evidence required to reconstruct the path, such as routing entries, interface states or forwarding-table outputs. Similar to RCA evidence score, let $\mathcal{A}_{\text{Path}}^*$ and $\hat{\mathcal{A}}_{\text{Path}}$ denote the golden evidence steps and the agent steps, respectively. We compute:

$$\text{Path-Evidence} = \text{F1}(\hat{\mathcal{A}}_{\text{Path}}, \mathcal{A}_{\text{Path}}^*). \quad (10)$$

4.3 Cost and Efficiency Metrics

We additionally report latency, interaction rounds, and token consumption. Token consumption is defined as:

$$\text{Tokens}_{\text{total}} = \text{Tokens}_{\text{input}} + \text{Tokens}_{\text{output}}. \quad (11)$$

These quantities are treated as cost and efficiency descriptors rather than substitutes for diagnostic capability.

Agent	RCA				Path Restoration			
	RCA Acc	RCA-Loc	RCA-ID	RCA-Evid.	Path Acc	Path-Loc	Path-Rest	Path-Evid.
Codex+GPT-5.5	47.62 ±4.5	52.90 ±3.7	66.83 ±3.7	15.80 ±1.0	87.96 ±3.2	99.38 ±0.6	95.28 ±1.4	47.84 ±1.5
ClaudeCode+Qwen3.7-Plus	19.84±3.6	34.68±3.4	38.96±3.8	12.62±0.8	17.59±3.7	87.65±2.8	48.19±3.0	26.58±1.6
HermesAgent+DeepSeek-V4-Pro	17.46±3.4	29.43±3.6	40.08±3.9	10.36±0.8	26.85±4.3	85.49±2.9	59.26±2.8	21.85±1.3
HermesAgent+Qwen3.7-Max	25.40±3.9	37.59±3.6	52.38±3.9	12.59±0.8	40.74±4.8	81.48±3.6	59.00±3.5	24.06±1.7
HermesAgent+TelecomGPT-R1	4.76±1.9	12.41±2.9	11.37±2.5	5.23±0.7	1.85±1.3	75.31±3.6	10.71±2.0	6.13±0.8

Table 5: Overall capability results. Scores are percentages and reported as mean $\pm$ standard deviation error.

5 Experiments and Analysis

We evaluate five representative agent-model combinations: Codex+GPT-5.5, ClaudeCode+Qwen3.7-Plus, HermesAgent+DeepSeek-V4-Pro, HermesAgent+Qwen3.7-Max, and HermesAgent+TelecomGPT-R1 [15]. All agents are evaluated on the same task set and are not given access to ground-truth answers or golden evidence steps during inference. Each task is executed in a reactive sandbox environment that exposes the permitted telecom diagnostic interface. Agents interact with the environment by issuing commands or tool calls and receiving observations. We log the full trajectory, including device queries, returned observations, final answers, latency, interaction rounds, and token usage. The same answer normalization and evidence-matching rules are applied across all agents. Full environment details, prompts, and code are provided in the supplementary material.

5.1 Overall Capability Results

Table 5 reports the main capability results. We report the task accuracy together with the capability metrics presented in Sec. 4: localization, identification, restoration, and evidence acquisition. In our experiments, agents perform better on path-restoration tasks than on RCA tasks. Codex+GPT-5.5 obtains the strongest overall results, and it performs especially well on path restoration.

The results show also that final accuracy alone hides important differences between agent capabilities. For example, Codex+GPT-5.5 reaches only 47.84% Path-restoration evidence, showing that producing a correct or plausible answer does not imply that the agent produces the evidence expected from a telecom troubleshooting engineer. Similarly, an agent (e.g., HermesAgent+DeepSeek-V4-Pro) may achieve high path-localization performance (Path-Loc) while still failing to restore the full path (Path Macro IoU) or collect sufficient diagnostic evidence (Path-Evidence). In addition, agents are better in RCA identification than in RCA localization e.g., 66.83% vs 52.9% for Codex+GPT-5.5 and 52.38% vs 37.59% for HermesAgent+Qwen3.7-Max, respectively, indicating that agents sometimes infer the right fault type but fail to identify the precise faulty node or object.

5.2 Cost and Efficiency

Table 6 reports interaction rounds, latency, and token consumption. These metrics are cost and efficiency descriptors, rather than capability measures. The results show that higher computational cost does not necessarily translate into stronger diagnostic capability. For instance, ClaudeCode+Qwen3.7-Plus consumes substantially more rounds and tokens than Codex+GPT-5.5, but obtains lower overall capability scores. In contrast, HermesAgent+Qwen3.7-Max outperforms HermesAgent+DeepSeek-V4-Pro in path restoration (see Path Acc) but at a cost of higher latency and token consumption. This motivates reporting cost alongside capability rather than using latency or token consumption as proxies for agent competence.

Remark (Human expert results). *As a reference, during the independent review phase (see Figure 1), human experts have achieved 92.6% and 56.4% accuracy on path-restoration tasks and RCA tasks, respectively. In addition, it took them between 40 and 60 minutes to complete each of these tasks.*

5.3 RCA Performance by Fault Category

Figure 3 visualizes RCA accuracy across the five root-cause categories presented in Table 3. The category-level results show that agent failures are not uniformly distributed across telecom fault

Agent	Task	Rds	Lat.(s)	Tokens
Codex+GPT-5.5	RCA	10.81	333.40	476.5k
Codex+GPT-5.5	Path	14.14	419.80	1019.2k
ClaudeCode+Qwen3.7-Plus	RCA	81.36	1234.64	2751.7k
ClaudeCode+Qwen3.7-Plus	Path	93.85	1241.09	3143.4k
HermesAgent+DeepSeek-V4-Pro	RCA	36.62	483.62	1491.7k
HermesAgent+DeepSeek-V4-Pro	Path	38.66	497.84	1453.1k
HermesAgent+Qwen3.7-Max	RCA	31.09	1352.93	1218.6k
HermesAgent+Qwen3.7-Max	Path	36.77	1482.64	1574.6k
HermesAgent+TelecomGPT-R1	RCA	31.76	641.77	656.2k
HermesAgent+TelecomGPT-R1	Path	35.53	934.40s	797.1k

Table 6: Efficiency and cost results.

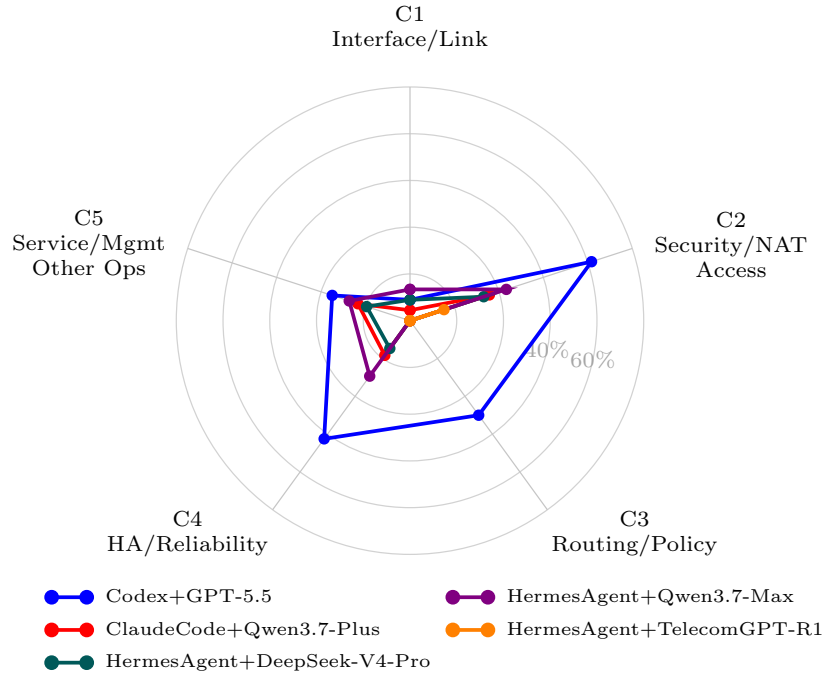


Figure 3: RCA accuracy across root-cause categories for the five evaluated agent-model combinations. The radar plot shows that agent performance varies substantially across fault families rather than shifting uniformly across all categories.

families. Two patterns holds across all agents. C2 faults are the most reliably diagnosed by every agent, and C1 is uniformly hard, with accuracy between 0.00% and 10.71%. The remaining ordering is agent-dependent. C3 separates the agents sharply: no agent other than Codex+GPT-5.5 solves a single C3 task, while Codex+GPT-5.5 reaches 40.00% and fails instead on C1. Codex+GPT-5.5 is also the only agent with a wide gap between C4 and C5 (50.00% versus 28.12%); the remaining agents differ by at most four points on these two categories. This suggests that CTBench does not only measure generic troubleshooting ability: it also reveals which telecom fault families remain challenging for current agents.

5.4 Impact of Observability and Heterogeneity

We next analyze how task metadata explains agent behavior. To save space, the main paper reports this analysis for ClaudeCode+Qwen3.7-Plus as a representative agent. Full results for all agents are provided in the supplemental material.

Observability. For ClaudeCode+Qwen3.7-Plus, RCA performance drops under partial observability. On fully observable RCA tasks, the agent obtains 20.56% accuracy, 37.02% localization, 42.92% RCA identification, and 38.29% evidence coverage. On partially observable tasks, accuracy decreases to 15.79%, localization drops to 17.14%, and RCA identification drops to 16.22%. Interestingly, evidence coverage increases to 62.20%, suggesting that the agent can still collect some indirect observations but struggles to convert them into precise fault localization and root-cause identification. This supports the role of observability as a structural difficulty factor: partial evidence does not merely slow the agent down; it changes the nature of the reasoning required.

Network heterogeneity. Network heterogeneity also affects performance, especially for path restoration. For ClaudeCode+Qwen3.7-Plus, path accuracy decreases from 29.31% in low heterogeneity settings to 4.00% in high heterogeneity settings. Path localization also decreases from 94.96% to 72.73%, while path restoration IoU drops from 68.58% to 35.05%. This indicates that heterogeneous multi-device settings make it harder for the agent to integrate evidence across device roles, vendor-specific command outputs, and forwarding contexts. For RCA, the effect is less uniform: accuracy decreases from 33.33% to 13.79%, while localization and RCA identification remain similar or slightly increase. This suggests that heterogeneity interacts with other factors such as fault category, observability, and protocol complexity, and should not be used alone to define task complexity.

6 Discussion

Performance is multi-faceted. A key lesson from CTBench is that agent performance should be measured in detailed ways: separating localization, identification, restoration, and evidence acquisition exposes distinct variations in abilities.

Evidence-grounded diagnosis remains difficult. Even when agents produce plausible final answers, they often do not produce evidences that telecom experts consider necessary. This is critical for operational trust, where a diagnosis must be justified before remediation.

Partial observability and heterogeneity expose agent limitations. Under partial observability, agents must reason from indirect evidence and cross-checks. Under heterogeneity, they must normalize device roles, vendor-specific commands, and output formats. The observed drops in localization, identification, and restoration show that these are not merely implementation details; they are core dimensions of telecom-agent capability.

CTBench reflects general agent challenges. Although focusing on telecom operations, CTBench presents broader agentic capabilities required in complex professional environments, including partial-observation reasoning, interaction with heterogeneous environment, causal attribution, and long-horizon evidence planning.

The supplementary material provides full capability analysis derived from trajectory-level failures.

7 Limitations

CTBench can be extended in future work. First, CTBench covers only two types of tasks: Path-Restoration and RCA. The capability of dynamically repairing faults in the network environment by the evaluation agent is not covered. This will be supplemented in the future. In addition, while the benchmark covers realistic telecom troubleshooting, it does not yet exhaustively cover all domains such as full radio-access-network operations, core-network slicing, or cloud-native telecom infrastructure.

8 Conclusion

We presented CTBench, an agentic benchmark for realistic telecom network operations and maintenance. Using with fine-grained data annotations and multi-dimensional metrics, CTBench assesses agent proficiency in fault localization, root-cause isolation, path reconstruction, expert-aligned

evidence extraction, and reasoning robustness under partial observability and heterogeneous environments. Experiments with representative agent-model combinations reveal that current agents remain far from reliable telecom troubleshooting operators, especially in evidence-grounded RCA and partially observable settings. We hope CTBench will support future research on trustworthy, domain-grounded, and operationally safe AI agents for communication networks.

References

- [1] Lina Bariah, Brahim Mefgouda, Farbod Tavakkoli, Enrique Molero, Louis Powell, and Merouane Debbah. Telcoagent-bench: A multilingual benchmark for telecom ai agents, 2026.
- [2] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -Bench: Evaluating conversational agents in a dual-control environment. In *Proceedings of the Forty-Third International Conference on Machine Learning (ICML)*, 2025.
- [3] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.
- [4] Pranshav Gajjar and Vijay K. Shah. ORAN-Bench-13K: An open source benchmark for assessing LLMs in open radio access networks. In *Proceedings of the 2025 IEEE 22nd Consumer Communications & Networking Conference (CCNC)*, pages 1–4, 2025.
- [5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [6] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [7] Ali Maatouk, Kenny Chirino Ampudia, Rex Ying, and Leandros Tassioulas. Tele-llms: A series of specialized large language models for telecommunications. *IEEE Access*, 14:86424–86441, 2026.
- [8] Ali Maatouk, Fadhel Ayed, Nicola Piovesan, Antonio De Domenico, Merouane Debbah, and Zhi-Quan Luo. TeleQnA: A benchmark dataset to assess large language models telecommunications knowledge. *IEEE Network*, 2025.
- [9] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for general AI assistants. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [10] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Hao Peng, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [11] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115:211–252, 2015.
- [12] Mohamed Sana, Nicola Piovesan, Antonio De Domenico, Yibin Kang, Haozhe Zhang, Merouane Debbah, and Fadhel Ayed. Reasoning language models for root cause analysis in 5g wireless networks, 2025.
- [13] Jingwen Tong, Zijian Li, Fangyu Liu, Wei Guo, and Jun Zhang. Wirelessagent++: Automated agentic workflow design and benchmarking for wireless networks, 2026.

- [14] Ahmed Twabi, Yepeng Ding, and Tohru Kondo. Netagentbench: A state-centric benchmark for evaluating agentic network configuration, 2026.
- [15] Bohao Wang, Chenwei Wu, Haoyu Li, Hang Zou, Yu Tian, Lina Bariah, Chongwen Huang, Yongliang Shen, Zhaoyang Zhang, and Mérouane Debbah. TelecomGPT-R1: Post-training recipes for universal reasoning in telecom. <https://huggingface.co/KU-DFI/TelecomGPT-R1>, 2026.
- [16] Zhihao Wang, Alessandro Cornacchia, Alessio Sacco, Franco Galante, Marco Canini, and Dingde Jiang. A network arena for benchmarking ai agents on network troubleshooting. In *Proceedings of the Fourteenth International Conference on Learning Representations (ICLR)*, 2026.
- [17] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.
- [18] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, Qi Zhang, and Tao Gui. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 68(2):121101, 2025.
- [19] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- [20] Yajie Zhou, Jiajun Ruan, Eric S. Wang, Sadjad Fouladi, Francis Y. Yan, Kevin Hsieh, and Zaoxing Liu. Netarena: Dynamic benchmarks for ai agents in network automation. In *Proceedings of the Fourteenth International Conference on Learning Representations (ICLR)*, 2026.

Supplementary Material for Paper:

CTBench: Evaluating Troubleshooting Capabilities of AI Agents in Realistic Telecom Network Operations

Table of Contents

A Dataset and Annotation Details	14
A.1 RCA task	14
A.2 Path Restoration task	15
A.3 Annotation Protocol	15
A.4 Metadata Schema	16
A.5 RCA Metadata Distributions	16
A.6 Path-Restoration Metadata Distributions	17
A.7 Task Examples	17
B Supplementary Evaluation Metrics	17
B.1 Hierarchical RCA Scoring	18
B.2 Topology-Aware Node Similarity	19
B.3 Root-Cause Type Hierarchy	19
C Supplementary Evaluation Results	19
C.1 Full RCA Category Results	19
C.2 Hierarchical RCA Results	19
C.3 Observability Results for All Agents	19
C.4 Heterogeneity Results for All Agents	19
C.5 Structural Difficulty Trends	20
C.6 Evidence-Grounded Performance Analysis	22
C.7 Harness Sensitivity Analysis	23
D Reproducibility Details	24
D.1 Evaluation Environment	24
D.2 Agent Harnesses and Concurrency	25
D.3 Command Form	25
D.4 Prompt Template	25
E Trajectory-Level RCA Failure Analysis	26
E.1 Overview of Trajectory-Level Failure Analysis	26
E.2 Failure Mode Taxonomy	27
E.3 Representative Failure Trajectories	27
E.4 Implications for Agent Design	28
F Discussion of Agent Capabilities in CTBench	29

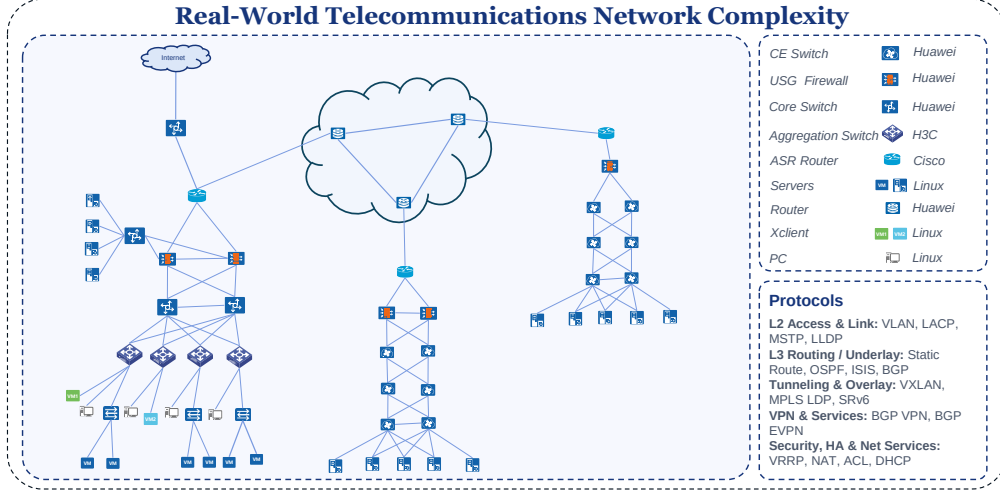


Figure 4: Representation of the telecom network under study with multi-vendor equipment and realistic networking protocols.

A Dataset and Annotation Details

CTBench is constructed through an expert-in-the-loop process. CTBench involves real network equipment with different vendors, equipment, protocols, and interfaces (see Figure 4). Candidate tasks are abstracted from realistic telecom maintenance cases and sanitized to remove sensitive production information. 15 senior telecom domain experts with an average of 20 years of professional experience define the problem statement, available device outputs, permitted command interface, standard answer, and task-side metadata. The current benchmark contains 126 RCA tasks and 108 path-restoration tasks.

A.1 RCA task

The RCA subset contains 126 tasks and 214 normalized root-cause triples. Since a single troubleshooting task may contain multiple independent faults, the number of root-cause entries can exceed the number of tasks. The dataset also contains 176 task-level root-cause mentions, where each root-cause type is counted at most once per task. The difference between 214 entries and 176 mentions reflects repeated affected objects or devices for the same root-cause type within some tasks.

The 17 normalized root-cause labels are grouped into seven operational fault domains for dataset characterization. The main paper merges sparse domains into C5 for stable reporting, while Table 7 reports the complete unmerged distribution.

Fault domain	Types	Entries	Ratio
Interface and link-state faults	1	60	28.04%
Security, NAT, and boundary access control	3	52	24.30%
Routing protocol and policy control	2	36	16.82%
High availability and reliability mechanisms	2	34	15.89%
Overlay and VPN service faults	4	26	12.15%
Basic addressing and Layer-2 access configuration	4	5	2.34%
Monitoring and operational visibility	1	1	0.47%
Total	17	214	100.00%

Table 7: Root-cause category coverage in CTBench fault-localization tasks.

Table 8 reports the fine-grained root-cause distribution. The distribution is intentionally not class-balanced: CTBench preserves frequent operational failure modes while retaining diagnostically distinct long-tail cases. We further merge the fine-grained root-causes into five-category taxonomy shown in Table 9.

Fine-grained root cause	Fault domain	Entries	Ratio	Task ment.
Shutdown	Interface/link state	60	28.04%	28
Security policy does not permit the user	Security/NAT/access control	41	19.16%	41
IP prefix list misses the corresponding source IP	Routing/policy control	24	11.21%	24
Global STP is not enabled	High availability/reliability	23	10.75%	23
L3VPN misconfiguration	Overlay/VPN service	17	7.94%	17
OSPF misconfiguration	Routing/policy control	12	5.61%	6
Global HRP hot-standby protocol is not enabled	High availability/reliability	11	5.14%	11
NAT outside-interface attribute error or omission	Security/NAT/access control	7	3.27%	7
NAT inside-interface attribute error or omission	Security/NAT/access control	4	1.87%	4
SRv6-Policy tunnel planning error	Overlay/VPN service	4	1.87%	4
VXLAN misconfiguration	Overlay/VPN service	4	1.87%	4
MAC address misconfiguration	Basic L2/access config	2	0.93%	2
Interface IP error	Basic L2/access config	1	0.47%	1
Interface VLAN misconfiguration	Basic L2/access config	1	0.47%	1
VPN configuration missing	Overlay/VPN service	1	0.47%	1
Host-information collection missing	Monitoring/visibility	1	0.47%	1
Loopback IP conflict	Basic L2/access config	1	0.47%	1
Total	—	214	100.00%	176

Table 8: Fine-grained root-cause distribution in CTBench. Task mentions count each root-cause type at most once per task.

Category	Entries	Ratio
C1 Interface/link state	60	28.04%
C2 Security/NAT/access	52	24.30%
C3 Routing/policy	36	16.82%
C4 HA/reliability	34	15.89%
C5 Service/management/other	32	14.95%

Table 9: Merged RCA category distribution over 214 gold root-cause entries.

A.2 Path Restoration task

The path-restoration subset contains 108 tasks. Each task asks the agent to reconstruct the forwarding path from a source endpoint to a destination endpoint or destination IP. Unlike RCA tasks, whose answers are sets of root-cause triples, path-restoration answers are ordered network paths. A valid answer must preserve hop order, path multiplicity, and physical egress-interface semantics. The required output uses one line per path, connects consecutive hops with $\rightarrow$, writes each non-terminal hop as `node_egress-interface`, and writes the terminal endpoint as the node name only.

Each golden path is decomposed into normalized node records with node names, egress interfaces, and raw output segments. This representation supports exact-match scoring while also enabling more diagnostic overlap measures, such as endpoint correctness, ordered edge overlap, interface overlap, and path-count correctness.

A.3 Annotation Protocol

For RCA tasks, standard answers are normalized into one or more minimal root-cause triples of the form $(fault_node, fault_object, root_cause)$. For path-restoration tasks, standard answers preserve node order, path multiplicity, and interface semantics. Golden solution paths are annotated as key evidence steps rather than complete command logs. Each retained step must correspond to necessary evidence acquisition, verification, or use. For multi-root-cause and multi-path tasks, reviewers check answer completeness and evidence sufficiency rather than enforcing a single unique reasoning trajectory. Cases with unresolved expert disagreement are revised or removed before release.

Metadata name	Fault localization	Path restoration
Golden answer	Normalized root-cause triples in <code>standard_answer.faults</code>	Ordered endpoint-to-endpoint paths in <code>standard_answer.paths</code>
Root-cause / path count	<code>data_labels.root_cause_count</code> ; <code>data_labels.has_multiple_root_causes</code>	<code>data_labels.restored_path_count</code> ; <code>data_labels.unique_node_count</code>
RCA category labels	<code>data_labels.rca_category_labels</code> ; <code>data_labels.rca_categories</code>	Not applicable
Evidence observability	<code>evidence_observability.tier</code> ; direct/degraded observation pairs	Not applicable
Fault-propagation chain	<code>fault_propagation_chain</code> . <code>total_propagation_step_count</code> ; causal propagation steps	Not applicable
Golden solution path	<code>golden_solution.path</code> . <code>golden_solution.length</code> ; diagnostic key steps	<code>golden_solution.path</code> . <code>golden_solution.length</code> ; path-reconstruction key steps
Network heterogeneity	<code>network_heterogeneity.level</code> ; vendor/type/device counts and key devices	<code>network_heterogeneity.level</code> ; vendor/type/device counts and key devices
Protocol complexity	<code>protocol_complexity</code> . <code>mechanism_count</code> ; root-cause, diagnostic, propagation, and observability mechanisms	<code>protocol_complexity</code> . <code>mechanism_count</code> ; forwarding, diagnostic, and path-restoration mechanisms

Table 10: Metadata fields for the two evaluated CTBench task types.

Evidence Observability			Root-Cause Count			Fault-Propagation Chain		
Tier	N	Ratio	Tier	N	Ratio	Tier	N	Ratio
O1 direct	107	84.92%	RC1	66	52.38%	Low (≤ 4)	19	15.08%
O2 indirect	19	15.08%	RC2	44	34.92%	Medium (5–13)	87	69.05%
			RC3+	16	12.70%	High (> 13)	20	15.87%
Golden Solution Path			Device Heterogeneity			Protocol Complexity		
Tier	N	Ratio	Tier	N	Ratio	Tier	N	Ratio
Short (≤ 4)	73	57.94%	H-Low ($=1$)	39	30.95%	Low/Mod. (≤ 7)	76	60.32%
Long (> 4)	53	42.06%	H-High (≥ 1)	87	69.05%	High (> 7)	50	39.68%

Table 11: RCA question-level component distributions over 126 tasks.

A.4 Metadata Schema

CTBench annotates each task with task-side metadata for dataset auditing, structural difficulty analysis, and post-hoc trajectory evaluation. These metadata fields are not provided to agents during inference. Table 10 summarizes the metadata fields used for the two evaluated task types.

A.5 RCA Metadata Distributions

Table 11 summarizes the question-level component distributions for the 126 RCA tasks. These metadata fields characterize observability, answer multiplicity, causal depth, evidence-path length, network heterogeneity, and protocol complexity.

The O2 subset contains tasks in which decisive evidence must be inferred from indirect observations. The multi-root-cause subset tests answer completeness. Fault-propagation-chain length captures causal depth, while golden solution length captures the number of expert-required evidence steps. Device heterogeneity and protocol complexity capture the operational diversity and cross-layer reasoning burden of the task.

Restored Path Count			Golden Solution Path		
Tier	N	Ratio	Tier	N	Ratio
P1 one path	64	59.26%	Short/Mod. (≤ 33)	74	68.52%
P2 two paths	20	18.52%	Long (> 33)	34	31.48%
P3+ three or more	24	22.22%			
Device Heterogeneity			Protocol Complexity		
Tier	N	Ratio	Tier	N	Ratio
H-Low (≤ 6)	58	53.70%	Low/Mod. (≤ 4)	48	44.44%
H-High (> 6)	50	46.30%	High (> 4)	60	55.56%

Table 12: Path-restoration question-level component distributions over 108 tasks.

A.6 Path-Restoration Metadata Distributions

The path-restoration subset contains 108 tasks. Table 12 summarizes the main path-restoration metadata distributions. Component counts are computed from normalized gold paths and metadata in the released task files.

Most path-restoration tasks have one gold path, but 44 tasks require multiple paths. These multi-path cases stress branching, redundancy, load balancing, and service-specific forwarding behavior. Path-restoration tasks also have longer evidence paths than RCA tasks because the agent must reconstruct complete forwarding behavior rather than identify one localized root cause.

A.7 Task Examples

We provide one RCA example and one path-restoration example to illustrate how task descriptions, gold answers, metadata, and golden evidence steps are connected. Agents receive only the task description and permitted tools during evaluation; they do not receive gold answers, metadata labels, or golden evidence steps.

RCA example. RCA-90 asks for the minimal set of root causes explaining why ping from Site1 area, Access-PC-01 to 20.1.1.10 is unreachable. The gold answer contains two port-fault entries: BoardLeaf-01;GE1/0/0;shutdown and BoardLeaf-01;GE1/0/1;shutdown. Both entries belong to C1 interface-state faults. The task is an RC2 task with two independent gold root causes. It is O1 because decisive interface-state evidence is directly observable on BoardLeaf-01. The golden solution path has three key steps, including interface description, current configuration, and LLDP neighbor checks. The task is H-Low because the normalized heterogeneity count is one device-role family.

Path-restoration example. PR-100 asks the agent to restore the path from SZ.Server.Cluster3 in the Shenzhen data center to SH.SAL_PC01 at 10.2.10.1 in the Shanghai branch. The gold answer contains one 12-element path from SZ.Server.Cluster3 through the Shenzhen core, PE devices, Beijing gateway, Shanghai aggregation, and finally SH.SAL_PC01. The task is a P1 task because the normalized gold answer contains one path. Its golden solution path has 33 expert key steps, placing it at the upper boundary of the Short/Moderate tier. Representative evidence includes route-table checks and LLDP-neighbor checks. The task is H-Low under the path-restoration threshold because its normalized heterogeneity count is six, and its protocol-complexity mechanism count is four.

B Supplementary Evaluation Metrics

The main paper reports strict IoU-based RCA localization and identification metrics. This section defines supplementary metrics used only for diagnostic error analysis. These metrics do not replace the primary scores.

Match level	Similarity	Interpretation
Exact label	1.0	The predicted normalized root-cause label is identical to the gold label.
Same subtype	0.8	The prediction falls under the same fine-grained failure subtype but misses the exact normalized label.
Same technical domain	0.6	The prediction belongs to the same operational domain, such as security/NAT access control, routing/policy control, or high-availability mechanisms.
Same super-category	0.4	The prediction remains within the same broad fault family, such as connectivity, policy, service, redundancy, or operational-visibility faults.
Unrelated	0.0	The prediction belongs to a different fault family or cannot be mapped to the root-cause taxonomy.

Table 13: Root-cause hierarchy similarity used by hierarchical RCA scoring.

Node relation	Similarity component	Interpretation
Exact node	1.0	The normalized predicted node is identical to the gold node.
Same logical cluster	0.7 taxonomy score	The nodes are in the same redundancy pair, HA pair, or inferred logical device cluster.
Same site and role	0.5 taxonomy score	The nodes share both site and device-role metadata.
Same site or same role	0.3 taxonomy score	The nodes share only one of site or role.
Topology distance 1	0.6 topology score	The nodes are one hop apart in the LLDP-derived topology graph.
Topology distance 2	0.3 topology score	The nodes are two hops apart in the LLDP-derived topology graph.
Disconnected or farther	0.0 topology score	The nodes are disconnected, unknown, or at distance at least three.

Table 14: Topology-aware node similarity used by hierarchical node location.

B.1 Hierarchical RCA Scoring

Strict RCA identification treats all incorrect root-cause labels equally. This binary treatment hides whether an incorrect label is outside the relevant mechanism family or only differs at a finer taxonomy. We therefore report a supplementary hierarchical RCA score based on the expert-defined RCA taxonomy in Table 13. The score gives partial credit when the predicted root-cause label matches the gold label at coarser levels of the taxonomy, such as the same subtype, technical domain, or broad fault family. This follows the common use of semantic hierarchies in classification analysis, for example in ImageNet-style label taxonomies [3, 11].

We use this score only for diagnostic analysis of failed trajectories. It separates predictions that are unrelated to the gold root cause from predictions that identify a related mechanism but choose the wrong normalized label. The primary benchmark score remains the strict exact-match metric; hierarchical RCA is reported as an auxiliary measure for interpreting near misses.

Let $\text{sim}(c, \hat{c})$ denote the similarity between a gold root-cause label c and a predicted label $\hat{c}$. We assign similarity 1.0 for an exact label match, 0.8 for the same fine-grained subtype, 0.6 for the same technical domain, 0.4 for the same super-category, and 0 otherwise. For multi-root-cause tasks, we define G_q as the set of gold RCA entries and P_q as the set of predicted RCA entries, compute all pairwise similarities between predicted and gold RCA entries, perform maximum-weight bipartite matching, and normalize the matched similarity by the number of gold RCA entries:

$$\text{Hier-RCA}(q) = \frac{\max_{M \in \mathcal{M}(P_q, G_q)} \sum_{(p, g) \in M} \text{sim}(c_g, c_p)}{|G_q|}. \quad (12)$$

B.2 Topology-Aware Node Similarity

We also define a topology-aware node similarity score as a supplement to exact node-object localization. Exact node matches receive full credit. Non-identical nodes receive partial credit when they belong to the same logical cluster, same site and role, or nearby topology neighborhood (see Table 14). This score evaluates node proximity only; it does not include object-level or root-cause-label similarity.

For non-identical nodes, the final hierarchical node similarity combines the taxonomy and topology components:

$$\text{node_sim} = 0.7 \cdot \text{taxonomy_sim} + 0.3 \cdot \text{topology_sim}. \quad (13)$$

If either node is absent from the LLDP-derived topology, the score falls back to the taxonomy component inferred from normalized node names, site, role, and redundancy metadata.

B.3 Root-Cause Type Hierarchy

Table 15 lists the normalized root-cause labels defined by the RCA output schema and their hierarchical grouping. The L0 column contains the complete set of schema-level root-cause labels that a model may output under the task instructions, including labels that do not appear as gold answers in the current 126-task release. L1, L2, and L3 provide progressively coarser groupings for near-miss analysis. Legacy gold-answer aliases are normalized to these schema-level L0 labels before hierarchical matching.

C Supplementary Evaluation Results

This section provides detailed result tables omitted from the main paper.

C.1 Full RCA Category Results

Table 16 reports localization, identification, and evidence scores for each model across the five merged RCA categories. These results complement the RCA category radar plot in the main paper.

C.2 Hierarchical RCA Results

Table 17 compares strict exact scores with supplementary hierarchical scores. Across the five evaluated agents, hierarchical RCA identification exceeds exact RCA identification by 2.65–13.14 percentage points, and hierarchical node localization exceeds exact localization by 16.28–31.24 points. These gaps measure the additional credit captured by the relaxed hierarchy when a prediction preserves a coarser root-cause category or topology neighborhood even though the strict normalized label or faulty object is incorrect.

The gap between exact localization and hierarchical node localization suggests that agents often reach the correct topology neighborhood but fail to identify the exact faulty node or object required for strict operational correctness.

C.3 Observability Results for All Agents

Table 18 reports RCA performance by observability level for all evaluated agents. The O2 subset contains tasks in which decisive evidence is unavailable or must be inferred from indirect observations. The O2→O1 rows correspond to an ablation setting where the originally unobservable elements are made directly observable. Since this ablation changes the golden evidence specification, we use it primarily to analyze final accuracy, localization, and identification rather than to draw conclusions from the evidence score.

C.4 Heterogeneity Results for All Agents

Table 19 reports performance by network heterogeneity level. Heterogeneity captures the need to reason across different device roles, vendors, command syntaxes, and output formats. The stratified results expose differences across heterogeneous settings that are hidden in aggregate performance.

L3 super-category	L2 technical domain	L1 root-cause subtype	L0 normalized root-cause label
Physical State / Link	Interface state	Administrative shutdown	shutdown
Physical State / Link	Interface performance	Bandwidth congestion	traffic congestion occupying port bandwidth
Physical State / Link	Interface MTU	MTU mismatch	MTU value misconfiguration
Security / Boundary Control	Security policy	Missing permit rule	security policy rule does not permit the corresponding user
Security / Boundary Control	NAT role binding	NAT external-interface attribute error	NAT external interface attribute misconfiguration or missing configuration
Security / Boundary Control	NAT role binding	NAT internal-interface attribute error	NAT internal interface attribute misconfiguration or missing configuration
Routing Control / Policy	Static route	Blackhole route	blackhole route
Routing Control / Policy	Static route	Missing static route	missing static route
Routing Control / Policy	Static route	Incorrect static route	incorrect static route
Routing Control / Policy	ARP / adjacency	ARP configuration	ARP configuration error
Routing Control / Policy	Forwarding loop	Layer-3 loop	Layer 3 loop
Routing Control / Policy	EGP control plane	BGP configuration	BGP configuration error
Routing Control / Policy	Route policy	Prefix-list coverage missing	IP prefix list missing the corresponding user source IP address
Routing Control / Policy	IGP control plane	OSPF configuration	OSPF configuration error
Routing Control / Policy	IGP control plane	ISIS configuration	ISIS configuration error
HA / Reliability	Layer-2 loop prevention	STP not enabled	global STP not enabled
HA / Reliability	Layer-2 loop prevention	Port STP not enabled	port STP not enabled
HA / Reliability	Hot-standby redundancy	VRRP/HRP redundancy not enabled	global VRRP hot standby redundancy protocol not enabled
Service / Overlay	VPN service isolation	L3VPN configuration	L3VPN configuration error
Service / Overlay	VPN service isolation	L2VPN configuration	L2VPN configuration error
Service / Overlay	VXLAN overlay	VXLAN configuration	VXLAN configuration error
Service / Overlay	SRv6 policy tunnel	SRv6-Policy planning	SRV6-Policy tunnel planning error
Service / Overlay	VPN service provisioning	Missing VPN configuration	VPN configuration missing
Access Configuration	Layer-2 forwarding identity	MAC address configuration	MAC address configuration error
Access Configuration	Interface addressing	Interface IP configuration	interface IP error
Access Configuration	VLAN access binding	Interface VLAN configuration	interface VLAN configuration error
Access Configuration	Address uniqueness	Loopback IP conflict	loopback IP configuration conflict
Operations Support	Host information collection	Missing host-information collection	host information collection function missing

Table 15: Root-cause type hierarchy used for hierarchical RCA scoring over the complete RCA output-schema label space.

C.5 Structural Difficulty Trends

We use task metadata to examine how performance changes across structurally easier and harder subsets. Because evidence observability and network heterogeneity are analyzed separately, Table 20

Agent	Category	RCA-Acc	RCA-Loc	RCA-ID	RCA-Evid.
Codex+GPT-5.5	C1	7.14	55.74	78.57	10.76
Codex+GPT-5.5	C2	65.31	80.33	80.65	13.45
Codex+GPT-5.5	C3	40.00	23.33	61.54	9.94
Codex+GPT-5.5	C4	50.00	51.43	51.43	18.50
Codex+GPT-5.5	C5	28.12	46.51	57.50	9.43
ClaudeCode+Qwen3.7-Plus	C1	3.57	40.30	60.61	9.77
ClaudeCode+Qwen3.7-Plus	C2	28.57	60.56	67.69	14.05
ClaudeCode+Qwen3.7-Plus	C3	0.00	0.00	10.00	4.49
ClaudeCode+Qwen3.7-Plus	C4	14.71	21.62	25.00	12.20
ClaudeCode+Qwen3.7-Plus	C5	18.75	36.96	37.78	9.32
HermesAgent+DeepSeek-V4-Pro	C1	7.14	39.39	57.58	9.06
HermesAgent+DeepSeek-V4-Pro	C2	26.53	55.56	61.02	7.78
HermesAgent+DeepSeek-V4-Pro	C3	0.00	0.00	31.94	6.48
HermesAgent+DeepSeek-V4-Pro	C4	11.76	13.89	13.89	11.23
HermesAgent+DeepSeek-V4-Pro	C5	15.62	32.50	35.90	7.04
HermesAgent+Qwen3.7-Max	C1	10.71	57.81	65.52	8.17
HermesAgent+Qwen3.7-Max	C2	34.69	72.73	80.33	12.64
HermesAgent+Qwen3.7-Max	C3	0.00	0.00	44.44	8.57
HermesAgent+Qwen3.7-Max	C4	23.53	21.05	35.29	12.70
HermesAgent+Qwen3.7-Max	C5	21.88	26.83	25.00	5.69
HermesAgent+TelecomGPT-R1	C1	0.00	12.90	26.67	3.85
HermesAgent+TelecomGPT-R1	C2	12.24	35.00	34.43	6.43
HermesAgent+TelecomGPT-R1	C3	0.00	0.00	0.00	2.03
HermesAgent+TelecomGPT-R1	C4	0.00	0.00	0.00	4.55
HermesAgent+TelecomGPT-R1	C5	0.00	0.00	0.00	3.50

Table 16: RCA results by fault category. Scores are percentages. C5 merges low-frequency service, overlay, configuration, management, and other operational categories.

Agent	Exact RCA-ID	Hier. RCA-ID	Exact Loc.	Hier. Node Loc.
Codex+GPT-5.5	66.50	72.52	52.69	73.80
ClaudeCode+Qwen3.7-Plus	39.59	50.19	35.03	58.70
HermesAgent+DeepSeek-V4-Pro	39.75	50.19	29.14	58.67
HermesAgent+Qwen3.7-Max	52.36	62.99	38.03	64.87
HermesAgent+TelecomGPT-R1	11.28	13.93	12.33	28.61

Table 17: Exact and hierarchical RCA scores. Scores are percentages. Hierarchical scores are recall-oriented over gold RCA entries and are supplementary diagnostics rather than replacements for exact IoU metrics.

focuses on root-cause count, restored path count, fault-propagation-chain length, golden-solution length, and protocol complexity.

Table 20 reports the average easiest-to-hardest Accuracy drop for each component. For RCA tasks, fault-propagation-chain length shows the largest drop: average Accuracy falls from 64.21% on FPC-Low cases to 7.00% on FPC-High cases. Root-cause count is also a strong factor, with RC1-to-RC3+ Accuracy decreasing by 31.63 points. Golden-solution length and protocol complexity show smaller but consistent drops across all five agents.

For path-restoration tasks, the drops are more balanced. Multiple restored paths, longer expert paths, and higher protocol complexity each reduce average Accuracy by roughly 20–23 points from the easy to hard tier. These metadata fields therefore separate easier and harder subsets for both task families, but should not be read as isolated causal variables, since each stratum still mixes different fault families, topologies, and evidence conditions.

Agent	Obs.	N	Acc.	RCA-Loc	RCA-ID	RCA-Evid.
Codex+GPT-5.5	O1	107	49.53	54.11	72.41	16.45
Codex+GPT-5.5	O2	19	36.84	42.86	35.48	12.11
Codex+GPT-5.5	O2→O1	19	73.68	81.82	75.00	14.08
ClaudeCode+Qwen3.7-Plus	O1	107	20.56	37.02	42.92	12.85
ClaudeCode+Qwen3.7-Plus	O2	19	15.79	17.14	16.22	11.55
ClaudeCode+Qwen3.7-Plus	O2→O1	19	31.58	37.50	25.71	11.66
HermesAgent+DeepSeek-V4-Pro	O1	107	17.76	29.55	45.05	9.99
HermesAgent+DeepSeek-V4-Pro	O2	19	15.79	28.57	15.00	12.37
HermesAgent+DeepSeek-V4-Pro	O2→O1	19	47.37	44.83	35.48	10.15
HermesAgent+Qwen3.7-Max	O1	107	23.36	37.85	53.04	12.11
HermesAgent+Qwen3.7-Max	O2	19	36.84	35.48	48.28	15.84
HermesAgent+Qwen3.7-Max	O2→O1	19	52.63	59.09	58.33	9.63
HermesAgent+TelecomGPT-R1	O1	107	5.61	14.00	13.43	5.80
HermesAgent+TelecomGPT-R1	O2	19	0.00	2.50	0.00	1.36
HermesAgent+TelecomGPT-R1	O2→O1	19	0.00	0.00	0.00	2.50

Table 18: RCA results by observability level and observability ablation. Scores are percentages. O2→O1 is evaluated on the same 19 originally O2 tasks after removing the unobservable elements.

Agent	Task	H	N	Acc.	Loc.	ID/Rest.	Evid.
Codex+GPT-5.5	RCA	H-Low (= 1)	39	66.67	52.73	57.69	12.47
Codex+GPT-5.5	RCA	H-High (> 1)	87	39.08	52.94	69.93	17.20
Codex+GPT-5.5	Path	H-Low (≤ 6)	58	89.66	100.00	95.52	38.89
Codex+GPT-5.5	Path	H-High (> 6)	50	86.00	98.02	95.11	53.29
ClaudeCode+Qwen3.7-Plus	RCA	H-Low (= 1)	39	33.33	30.38	34.25	11.76
ClaudeCode+Qwen3.7-Plus	RCA	H-High (> 1)	87	13.79	36.24	40.91	12.96
ClaudeCode+Qwen3.7-Plus	Path	H-Low (≤ 6)	58	29.31	94.96	68.58	25.72
ClaudeCode+Qwen3.7-Plus	Path	H-High (> 6)	50	4.00	72.73	35.05	27.14
HermesAgent+DeepSeek-V4-Pro	RCA	H-Low (= 1)	39	28.21	28.57	26.09	8.12
HermesAgent+DeepSeek-V4-Pro	RCA	H-High (> 1)	87	12.64	29.69	45.66	11.05
HermesAgent+DeepSeek-V4-Pro	Path	H-Low (≤ 6)	58	44.83	91.38	73.46	20.32
HermesAgent+DeepSeek-V4-Pro	Path	H-High (> 6)	50	6.00	72.41	50.77	22.91
HermesAgent+Qwen3.7-Max	RCA	H-Low (= 1)	39	46.15	38.71	42.86	9.57
HermesAgent+Qwen3.7-Max	RCA	H-High (> 1)	87	16.09	37.27	55.84	13.97
HermesAgent+Qwen3.7-Max	Path	H-Low (≤ 6)	58	48.28	91.38	72.38	19.72
HermesAgent+Qwen3.7-Max	Path	H-High (> 6)	50	32.00	67.92	50.28	26.87
HermesAgent+TelecomGPT-R1	RCA	H-Low (= 1)	39	10.26	8.57	5.97	2.98
HermesAgent+TelecomGPT-R1	RCA	H-High (> 1)	87	2.30	13.64	13.30	6.04
HermesAgent+TelecomGPT-R1	Path	H-Low (≤ 6)	58	3.45	83.19	17.97	8.96
HermesAgent+TelecomGPT-R1	Path	H-High (> 6)	50	0.00	58.68	5.24	4.18

Table 19: Selected results by network heterogeneity. Scores are percentages.

C.6 Evidence-Grounded Performance Analysis

In this subsection, evidence coverage is defined as the fraction of expert golden evidence steps observed in the trajectory, $|\hat{\mathcal{A}} \cap \mathcal{A}^*|/|\mathcal{A}^*|$. This recall-style quantity measures how much of the expert diagnostic chain the agent reaches, without penalizing additional commands. It is therefore different from the main evidence score used in the capability tables, which is an Evidence F1 score that accounts for both covered golden steps and extra executed device-command observations.

Table 21 reports evidence coverage on exactly solved tasks. Even when agents produce the correct final answer, their evidence coverage remains limited, suggesting that current agents often solve tasks without reconstructing the complete expert diagnostic chain. The table is conditioned on solved tasks, whereas Figure 5 analyzes all model-question samples.

Component	Task	Easy Acc.	Hard Acc.	Drop	Agents
Fault Propagation Length	RCA	64.21	7.00	57.21	5/5
Root-cause Count	RCA	37.88	6.25	31.63	5/5
Golden Solution Length	RCA	27.67	16.60	11.07	5/5
Protocol Complexity	RCA	26.84	17.20	9.64	5/5
Restored Path Count	Path	41.25	20.00	21.25	5/5
Golden Solution Length	Path	42.16	19.41	22.75	5/5
Protocol Complexity	Path	46.25	26.00	20.25	5/5

Table 20: Accuracy impact of CTBench components excluding evidence observability and network heterogeneity. Easy/Hard Accuracy is averaged over agents using the easiest and hardest tier of each component; Drop is the mean per-agent easy-to-hard Accuracy decrease in percentage points.

Agent	RCA	Path Restoration
Codex+GPT-5.5	58.05%	53.45%
ClaudeCode+Qwen3.7-Plus	51.45%	38.56%
HermesAgent+DeepSeek-V4-Pro	38.02%	19.61%
HermesAgent+Qwen3.7-Max	40.22%	27.75%
HermesAgent+TelecomGPT-R1	16.67%	39.13%

Table 21: Evidence coverage among exactly solved tasks.

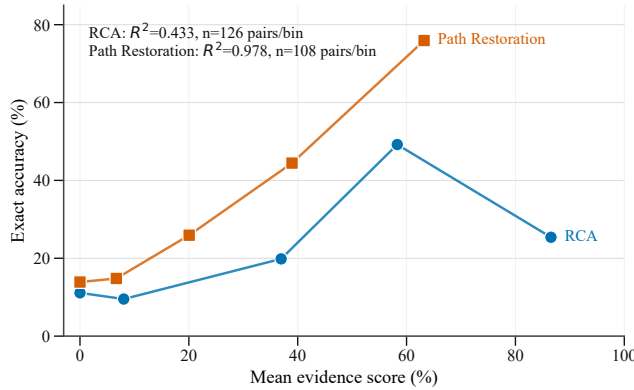


Figure 5: Evidence-accuracy relationship using equal-frequency quintiles sorted by evidence score. Each point aggregates the same number of model-question samples within a task family; the reported R^2 values are computed over the quintile points. The lines connect quintile aggregates and are not fitted regression lines.

Figure 5 further relates evidence coverage to exact-answer Accuracy after collecting all trajectories. Path Restoration shows a strong monotonic trend: higher evidence coverage corresponds to substantially higher exact Accuracy. RCA exhibits a weaker relationship; the lower RCA R^2 indicates that evidence coverage explains only part of the variance in exact correctness. This weaker relationship is consistent with the RCA error analysis: even when an agent reaches high evidence coverage, it may still fail to identify the correct fault node, affected object, root-cause label, or complete multi-fault answer. Thus, evidence acquisition is necessary for reliable troubleshooting, but RCA also requires stronger causal reasoning and answer normalization.

C.7 Harness Sensitivity Analysis

To separate model choice from execution-harness effects, we compare raw trajectory sets that use the same DeepSeek-V4-Pro model but different agent harnesses: ClaudeCode, HermesAgent, and Codex. This analysis treats each harness-model pair as an executable agent because the harness controls tool scheduling, state serialization, stopping behavior, and final-answer extraction. All runs

use the same task set, gold answers, and scoring scripts. The comparison is therefore intended as a raw harness sensitivity analysis under a fixed model family, rather than as an ablation that isolates each individual harness mechanism. Missing or empty final answers are kept in the denominator and counted as incorrect.

Task	Harness + Model	Acc.	Loc.	ID/Rest.	Evid.	Runtime	Tokens
RCA	ClaudeCode+DeepSeek-V4-Pro	21.43	26.19	34.85	37.71	410.52s	3809.7k
RCA	HermesAgent+DeepSeek-V4-Pro	17.46	29.43	40.08	33.38	481.13s	1750.8k
RCA	Codex+DeepSeek-V4-Pro	15.08	21.45	34.26	30.88	482.79s	1083.7k
Path Restoration	ClaudeCode+DeepSeek-V4-Pro	14.81	77.59	58.51	26.35	416.66s	5440.1k
Path Restoration	HermesAgent+DeepSeek-V4-Pro	26.85	81.90	59.26	18.84	492.97s	2155.5k
Path Restoration	Codex+DeepSeek-V4-Pro	26.85	77.59	59.25	24.88	545.00s	1660.8k

Table 22: Harness sensitivity for the same DeepSeek-V4-Pro model. Scores are percentages except runtime and tokens. ID/Rest. denotes RCA identification IoU for RCA and restoration-edge IoU for Path Restoration.

Table 22 shows that CTBench measures a harness-model combination rather than a model in isolation. On RCA, ClaudeCode+DeepSeek-V4-Pro obtains the highest exact Accuracy and evidence coverage among the three harnesses, while HermesAgent+DeepSeek-V4-Pro obtains the highest localization and identification IoU. Codex+DeepSeek-V4-Pro uses the fewest tokens and rounds, but its RCA exact Accuracy and evidence coverage are lower. On Path Restoration, HermesAgent and Codex reach the same exact Accuracy, whereas HermesAgent has the best localization IoU and Codex achieves nearly the same restoration-edge IoU with substantially fewer tokens and rounds than HermesAgent. ClaudeCode collects the most evidence for Path Restoration but uses far more tokens and obtains lower exact Accuracy. Runtime, rounds, and tokens should be interpreted as complementary efficiency measures: fewer interaction rounds or fewer tokens do not necessarily imply lower wall-clock latency because model-side response time and harness orchestration overhead can differ.

The disagreement counts in Table 23 show that different harnesses solve partly different subsets of tasks even when the underlying model is fixed. For example, on RCA, ClaudeCode solves 11 tasks that Codex misses, while Codex solves 3 tasks that ClaudeCode misses. On Path Restoration, Codex solves 19 tasks missed by ClaudeCode, while HermesAgent and Codex each solve 18 tasks missed by the other. Overall, these results show that the harness matters even when the underlying model is fixed. Different harnesses solve different subsets of tasks, so we report performance at the harness-model level rather than treating the model as the only source of variation.

D Reproducibility Details

D.1 Evaluation Environment

All CTBench agent trajectories are executed on a single orchestration host running EulerOS 2.0. The host is equipped with an Intel Xeon Gold 6230N CPU at 2.30GHz, 40 physical cores, 80 logical processors, and 502GB of physical memory. The host runs the agent harnesses, local sandbox workspaces, simulator/tool servers, logging, and result collection. Model inference is not served locally; all evaluated models are accessed through their original vendor APIs.

We pre-collected command-line interface (CLI) logs from the target network devices and served them through a mock backend. For supported commands, the backend returns the recorded device outputs; for invalid queries, it returns the corresponding error messages. Commands that are syntactically valid but absent from the collected logs return a simulated “permission denied” response, so agents cannot query outside the predefined observation set.

We evaluate five harness-LLM configurations with fixed harness versions: Codex (v0.141.0) with GPT-5.5, ClaudeCode (v2.1.63) with Qwen3.7-Plus, and Hermes Agent (v0.16.0) with DeepSeek-V4-Pro, Qwen3.7-Max, or TelecomGPT-R1. Each benchmark task runs in a fresh Docker container. The workspace, tool configuration, logs, and temporary files are recreated for each task to avoid state carryover between runs.

Task	Pair	Both	First only	Second only	Neither
RCA	CC vs. HA	15	12	7	92
RCA	CC vs. CX	16	11	3	96
RCA	HA vs. CX	12	10	7	97
Path Rest.	CC vs. HA	6	10	23	69
Path Rest.	CC vs. CX	10	6	19	73
Path Rest.	HA vs. CX	11	18	18	61

Table 23: Pairwise exact-answer disagreement under the same DeepSeek-V4-Pro model. CC, HA, and CX denote ClaudeCode, HermesAgent, and Codex, respectively. “First only” and “Second only” refer to the order in the Pair column.

D.2 Agent Harnesses and Concurrency

We evaluate three harness implementations: Codex, ClaudeCode, and HermesAgent. The reported agent-model settings are instantiated from these harnesses:

- Codex+GPT-5.5,
- ClaudeCode+Qwen3.7-Plus,
- HermesAgent+DeepSeek-V4-Pro,
- HermesAgent+Qwen3.7-Max, and
- HermesAgent+TelecomGPT-R1.

During evaluation, the task queue is capped at three concurrent agent runs to control API request pressure and avoid local simulator contention and the maximum running time for each initiated task is 3600 seconds.

D.3 Command Form

Each harness run follows the same orchestration pattern: select the harness and model, provide the CTBench task JSON, write trajectories and parsed results to a run-specific output directory, and set the maximum concurrency.

```
python <runner>.py --harness codex --model gpt-5.5
--input <ctbench_task_json> --output <result_json>
--workspace-base <sandbox_root> --concurrency 3 --timeout 3600
```

```
python <runner>.py --harness claudecode --model qwen3.7-plus
--input <ctbench_task_json> --output <result_json>
--workspace-base <sandbox_root> --concurrency 3 --timeout 3600
```

```
python <runner>.py --harness hermesagent --model <model_name>
--input <ctbench_task_json> --output <result_json>
--workspace-base <sandbox_root> --concurrency 3 --timeout 3600
```

For the Codex harness, the per-task child process created by the runner uses the following command shape, with the task prompt supplied on standard input:

```
codex.cmd -a never exec --cd <workspace_dir> --skip-git-repo-check
--sandbox workspace-write --color never --output-last-message last_msg.txt
--model gpt-5.5 --ephemeral
```

D.4 Prompt Template

All agent-model combinations use the same task-level prompt template shown in Figure 6. The prompt contains the task description, question identifier, allowed tool interface, and required answer format. It does not include ground-truth answers, task metadata, root-cause categories, difficulty labels, or golden evidence steps.

Agent Prompt Template

You are an AI agent evaluating comprehensive capabilities for CTBench. Your task is to solve the given CTBench evaluation problem using only the information and tools allowed by the task. Follow the task-specific instructions exactly. The task description defines the valid fault categories, answer fields, separators, line order constraints if any, device names, destination formats, and whitespace requirements. These task-specific requirements override any general wording in this prompt.

Work method:

1. Identify the affected endpoints, objects, and evaluation scope from the task.
2. Query only relevant devices and commands needed to confirm or reject plausible root causes.
3. Prefer direct evidence and cross-checking evidence from the task-allowed command outputs.
4. Interpret potential faults in the context of the intended network design; do not report a root cause that contradicts the topology, configuration model, or design logic implied by the task.
5. Do not perform broad exhaustive sweeps when a focused path can answer the task. Avoid repeated equivalent commands after the evidence is sufficient.
6. If multiple independent root causes exist, return the minimal complete set of root causes. Do not include downstream symptoms or duplicate faults.
7. If the task restricts the fault type, output only faults of the allowed type.
8. Use exact device names, interface names, destination IPs, object names, and root-cause labels from the task and observed outputs.

Evidence and safety rules:

- Do not use hidden ground truth, answer files, prior trajectory files, scoring scripts, or local simulator output files.
- Do not infer answers from filenames, dataset metadata, process state, or filesystem paths.
- Query simulator device outputs only through the API or tool interface explicitly allowed by the task.
- If an allowed command is unavailable, use the closest allowed command form from the task description; do not treat a missing command output as direct proof of a fault.

Before finalizing, silently check:

- Does every output line match the exact format requested by the task?
- Are all required fields present and in the requested order?
- Are separators exactly as requested, usually English semicolons?
- Are there no extra spaces, comments, labels, markdown, or explanations?
- Are all independent root causes included, and are non-root-cause symptoms excluded?

Put only the final answer inside `<result></result>` tags.

Task: {question_description}

Question ID: {question_id}

Return format: `<result>`your final answer here`</result>`

Figure 6: Agent Prompt Template.

E Trajectory-Level RCA Failure Analysis

E.1 Overview of Trajectory-Level Failure Analysis

This appendix summarizes a trajectory-level post-hoc analysis of exact-match RCA failures for Codex+GPT-5.5 and HermesAgent+Qwen3.7-Max. The goal is to characterize why an agent’s final answer fails after interacting with the diagnostic environment, rather than only reporting whether the final answer matches the gold RCA tuple. Among the 126 RCA tasks, Codex+GPT-5.5 has 66 exact-match failures and HermesAgent+Qwen3.7-Max has 94 exact-match failures; 60 tasks are failed by both systems.

The main failure pattern is not failed tool use. In many cases, agents issue relevant commands and observe useful intermediate evidence, but still fail when mapping that evidence to the final RCA tuple set. The observed errors include missing independent causes, adding non-minimal causes,

Level	Leaf failure label	Unit	Definition
Output-level	F5. Answer Realization Failure	Whole answer	The trajectory does not produce a valid final RCA answer, or the answer cannot be parsed into the required tuple schema.
Set-level	F1. Causal Coverage Deficit	Answer set	The answer omits one or more independent gold root causes, including dual-root, multi-device, or multi-interface faults.
Set-level	F2. Causal Minimality Violation	Answer set	The answer includes redundant, symptom-level, or non-independent causes beyond the minimal gold root-cause set.
Entry-level	F4. Mechanism Discrimination Error	Root-cause entry	The returned entry uses the wrong causal mechanism, such as confusing redundancy, routing, policy, address, or forwarding-layer faults.
Entry-level	F3. Fault-Object Grounding Error	Root-cause entry	The mechanism is largely correct, but the causal object is grounded to the wrong device, interface, IP address, policy object, underlay/overlay layer, or active/standby role.

Table 24: Hierarchical RCA failure taxonomy used for trajectory-level error analysis. Rows follow the annotation priority order.

choosing the wrong fault mechanism, grounding the fault to the wrong node or object, and violating the required output schema.

We identify this pattern through manual inspection of the diagnostic traces, checking each failed task against the task statement, final answer, issued commands, and observed command outputs.

E.2 Failure Mode Taxonomy

We organize trajectory failures using the hierarchical RCA failure taxonomy in Table 24. The taxonomy contains one output-level failure, two answer-set-level failures, and two entry-level failures. This taxonomy is used only as a post-hoc explanation of exact-match errors; it is not a formal component of the benchmark, is not part of the primary benchmark score, and does not relax the exact-answer criterion. This structure is important for reproducible annotation: when a prediction both misses an independent root cause and mislocalizes a returned tuple, the set-level coverage error is assigned first; entry-level mechanism or grounding errors are used only when the answer set is otherwise comparable to the gold set.

For each task, three expert annotators with telecom RCA expertise independently assigned failure labels, and the taxonomy was applied with a fixed priority order. Disagreements were resolved through adjudication and discussion under the same priority rules. The annotators first checked whether the final answer was parseable under the required RCA schema. If not, the failure was labeled as answer realization. For parseable answers, they next compared the predicted and gold root-cause sets to identify missing independent causes or non-minimal extra causes. Only after the answer set was comparable did they label entry-level failures: mechanism discrimination errors when the causal mechanism was wrong, and fault-object grounding errors when the mechanism was largely correct but the device, interface, address, policy object, or topology role was incorrect. Within entry-level annotation, mechanism mismatch has priority over object mismatch: a prediction with both a wrong mechanism and a wrong object is labeled F4 rather than F3.

Table 25 reports the resulting distribution. Coverage deficits and mechanism discrimination dominate Codex+GPT-5.5’s RCA errors, while HermesAgent+Qwen3.7-Max shows a more distributed error profile with substantially more minimality and answer-realization failures.

E.3 Representative Failure Trajectories

The full trajectories for these examples are provided in the Code and Data Supplement under `model_trajectory`, organized by model name, task type, and task identifier. All representative

Model	F1	F2	F3	F4	F5	Total
GPT-5.5	28	1	13	23	1	66
Qwen3.7-Max	32	16	18	16	12	94

Table 25: Failure-mode counts for exact-match RCA failures. F1–F5 refer to the labels in Table 24; each failed trajectory is assigned one primary failure label.

examples below were checked against the recorded final answers and diagnostic traces of the corresponding agents.

F5 Answer realization failure: RCA q12. The gold answer is FW_02;114.114.114.114;global VRRP hot standby redundancy protocol not enabled. HermesAgent+Qwen3.7-Max returns an empty final answer despite successfully interacting with the environment. The trajectory lacks valid result tags, separators, or parseable lines, leading to a complete output failure independent of the agent’s actual domain knowledge.

F1 Causal coverage deficit: RCA q7. The gold answer contains two independent root causes: FW_01;10.3.10.1;security policy rule does not permit the corresponding user and BJHQ_CSR1000V_GW_01;10.1.120.251;IP prefix list missing the corresponding user source IP address. Codex+GPT-5.5 outputs only the firewall security-policy root cause and omits the prefix-list fault, prematurely halting its diagnosis once the first locally plausible cause is identified.

F2 Causal minimality violation: RCA q18. The gold answer contains a VRRP redundancy configuration fault on FW_02, while HermesAgent+Qwen3.7-Max returns both a prefix-list fault and a VRRP fault on a different firewall. Although this answer also contains localization errors, the set-level F2 label takes precedence because the final submitted set retains unpruned, non-gold causal assertions alongside downstream symptoms.

F4 Mechanism discrimination error: RCA q10. The gold root cause is FW_02;114.114.114.114;global VRRP hot standby redundancy protocol not enabled. Codex+GPT-5.5 localizes the answer to the correct firewall and address but incorrectly reports an OSPF configuration error, successfully grounding the location while confusing redundancy-state semantics with routing-protocol configurations.

F3 Fault-object grounding error: RCA q11. The gold root cause is FW_02;8.8.8.8;global VRRP hot standby redundancy protocol not enabled. HermesAgent+Qwen3.7-Max outputs Core_SW_01;8.8.8.8;global VRRP hot standby redundancy protocol not enabled. The mechanism and address are aligned with the gold answer, but the causal device is misattributed to a neighboring core switch rather than the faulty firewall itself. The agent fails to decouple its observation point from the actual causal node.

E.4 Implications for Agent Design

The failure taxonomy suggests several checks that are useful for troubleshooting agents. During a run, the agent should track which symptoms have been explained, which candidate causes remain possible, which observations support or reject each candidate, and whether the final answer satisfies the required schema.

The observed failure modes suggest several useful checks for agent design:

- **Causal Coverage (F1) & Minimality (F2):** A global diagnostic state tracker may help ensure all symptoms are explained without including redundant downstream effects.
- **Semantic Grounding (F3 & F4):** Domain-specific verification may reduce mechanism confusion and mislocalization by checking protocol constraints (e.g., routing vs. redundancy states) and maintaining role-aware topology mapping to separate observation nodes from causal origins.

Failure taxonomy	General agent capability	CTBench property
F1 Causal coverage deficit	Causal attribution and long-horizon evidence planning	<code>root_cause_count</code> ; <code>golden_solution_path</code>
F2 Causal minimality violation	Causal attribution and symptom pruning	<code>fault_propagation_chain</code> ; <code>golden_solution_path</code>
F3 Fault-object grounding error	Interaction with heterogeneous environments	<code>network_heterogeneity</code>
F4 Mechanism discrimination error	Partial-observation reasoning and causal attribution	<code>protocol_complexity</code> ; <code>evidence_observability</code>
F5 Answer realization failure	Reliable structured answer realization	Output schema constraints

Table 26: Qualitative mapping from CTBench RCA failure modes to broader agent capabilities and related task properties.

- **Robust Finalization (F5):** Deterministic output validation can enforce schema constraints before final submission.

Overall, this fine-grained analysis suggests that, in these RCA failures, current agents more often struggle with multi-step diagnostic reasoning and causal minimization than with basic tool execution.

F Discussion of Agent Capabilities in CTBench

Although CTBench is grounded in telecom network operations, the trajectory-level RCA failures reflect broader agent capabilities required in complex professional environments. The goal of this discussion is to clarify how the failure modes summarized in Table 24 relate to general agent capabilities and how these pressures arise from CTBench task characteristics. In many failed trajectories, agents can collect relevant observations but still fail to maintain causal coverage, prune downstream symptoms, ground evidence to the correct object, distinguish similar mechanisms, or realize the final answer in the required structured format.

These failure modes are consistent with the task properties of CTBench RCA: heterogeneous device roles, partial observability, layered protocol mechanisms, causal propagation, multiple independent faults, and multi-step evidence requirements. Table 26 summarizes how the observed failure modes relate to broader agent capabilities and the CTBench properties that stress them.